

VibeTrader Repository Report

Date: 2026-05-03 | Repository: vibetrader | Branch inspected: add-repo-report-pdf-e2ffb2

Executive summary. VibeTrader is a research-oriented Python project that experiments with using an image-editing diffusion model to forecast short-term cryptocurrency chart movement. Instead of predicting numeric prices directly, it renders OHLCV data as candlestick-chart images, fine-tunes an InstructPix2Pix/Stable Diffusion pipeline to extend those charts by four candles, and then translates the generated image into a BUY, SELL, or HOLD signal.

The repository contains the end-to-end concept: data ingestion, chart rendering, dataset assembly, training helpers, inference, signal extraction, backtesting, a paper-trading loop, and a Flask-based interactive UI. Its largest practical limitation is that several required runtime artifacts are excluded from Git, so a fresh clone is not immediately runnable.

Project Purpose

1. Convert market time series into chart images rather than feed raw numbers into a classic forecasting model.
2. Fine-tune a diffusion image model to draw plausible future candles.
3. Extract a discrete trading signal from the generated continuation.
4. Evaluate whether that signal can produce useful simulated returns.

This makes the repository better described as an applied ML research prototype than a production trading system.

Architecture

Stage	Primary files	Responsibility
Data ingestion	<code>data/fetch_ohlcv.py</code>	Downloads OHLCV candles via <code>ccxt</code> and computes RSI/MACD features.
Image generation	<code>data/render_charts.py</code>	Renders 256x256 candlestick charts and labels target samples with BUY/SELL/HOLD markers.
Dataset assembly	<code>data/build_dataset.py</code>	Builds a Hugging Face dataset for InstructPix2Pix fine-tuning.
Training	<code>train/run_training.sh</code> , <code>train/run_training_gpu.sh</code>	Launches Stable Diffusion / InstructPix2Pix fine-tuning with different hardware assumptions.
Inference	<code>inference/predict.py</code>	Loads the fine-tuned pipeline and generates chart continuations.
Signal extraction	<code>inference/extract_signal.py</code> , <code>inference/extract_signal_mistral.py</code>	Turns generated images into BUY/SELL/HOLD decisions using either pixel heuristics or a Mistral vision model.
Evaluation	<code>bot/backtest.py</code>	Backtests the model on held-out data and computes simulated returns.
Interactive serving	<code>app.py</code> , <code>frontend-v3/index.html</code>	Serves a simple interactive UI over Flask.

Workflow summary: fetch OHLCV -> render chart pairs -> build dataset -> train diffusion model -> predict future candles -> extract signal -> backtest or inspect through the UI.

Major Directories and Files

Core code

Artifacts and docs

- `app.py` : Flask backend and API.
- `bot/` : backtesting and paper trading.
- `data/` : market fetching, feature engineering, chart rendering, dataset building.
- `inference/` : model loading, prediction, signal extraction, checkpoint comparison.
- `train/` : environment setup and training launchers.
- `README.md` : strong narrative and quickstart.
- `outputs/` : saved backtest metrics, CSVs, and comparison samples.
- `demo.ipynb` : notebook exploration.
- `plot_pnl_distribution.py` and `pnl_distribution.png` : trade distribution analysis.
- `requirements.txt` : primary runtime dependencies.

Dependencies

- **ML stack:** `torch`, `diffusers`, `transformers`, `accelerate`, `safetensors`
- **Data and processing:** `pandas`, `numpy`, `datasets`, `tqdm`
- **Market data and indicators:** `ccxt`, `pandas-ta`
- **Rendering and charts:** `Pillow`, `matplotlib`, `mplfinance`
- **Tracking and serving:** `wandb`, `flask`, `mistralai`

Operationally, the repository also expects external model weights, local checkpoint artifacts, and optionally secrets such as `MISTRAL_API_KEY`, `WANDB_API_KEY`, `HF_TOKEN`, and Binance credentials.

Operational Workflow

1. Fetch raw market candles into CSV.
2. Compute indicators and render sliding chart windows.
3. Assemble a Hugging Face dataset for image-edit training.
4. Fine-tune the diffusion model.
5. Run inference on held-out or live windows.
6. Convert the generated chart into a discrete signal.
7. Measure results through backtesting or view predictions in the Flask UI.

The workflow is coherent, but it depends on heavyweight generated directories that are not stored in the repository.

Strengths

- Distinctive research angle with a clearly explained thesis.
- End-to-end prototype coverage from data collection through UI.
- Readable, compact codebase with intuitive file boundaries.
- Committed sample datasets and outputs make inspection easier.
- Two signal extraction modes create useful comparison opportunities.
- Simple Flask plus static HTML delivery surface.

Risks and Notable Gaps

Fresh-clone operability

The repository does not include `checkpoints/`, `data/rendered/`, or `data/dataset/`. Those paths are ignored and absent in this checkout, but the app and several scripts require them. A new operator cannot run the full stack immediately after cloning.

Stale code paths

`extract_signal.py` returns `action`, `confidence`, `green_pct`, and `red_pct`. However, `bot/trader.py`, `inference/generate_comparison.py`, and notebook snippets still reference `signal.avg_rgb`, which no longer exists. Those paths will fail when exercised.

Live trading fidelity

The paper trader does not compute current RSI/MACD values. It hardcodes `RSI=50.0` and `MACD=0.0`, which makes live prompting inconsistent with the training and backtest pipeline.

Evaluation realism

Backtesting does not model fees, slippage, spread, latency, or risk controls. That is acceptable for exploratory research, but it reduces confidence that reported returns would survive real execution.

Engineering maturity

The repo has no visible automated tests, CI workflow, lockfile, container definition, or artifact bootstrap path for local inference. There is also some ambiguity between tracked generated outputs and ignored parent directories, which can complicate contributor expectations.

Overall Assessment

VibeTrader is a compact and interesting ML research prototype with a strong narrative and a credible experimental pipeline. It is best understood as a proof-of-concept laboratory for visual financial forecasting rather than a production trading application.

Its strongest qualities are clarity of idea and breadth of prototype coverage. Its weakest areas are reproducibility, runtime completeness, and production hardening. Addressing missing artifacts and the stale code references would materially improve the repository's usability.

Files Consulted

`README.md`, `requirements.txt`, `app.py`, `data/fetch_ohlcv.py`, `data/render_charts.py`, `data/build_dataset.py`, `inference/predict.py`, `inference/extract_signal.py`, `inference/extract_signal_mistral.py`, `inference/generate_comparison.py`, `bot/backtest.py`, `bot/trader.py`, `train/run_training.sh`, `train/run_training_gpu.sh`, `train/brev_setup.sh`, `outputs/BACKTEST_REPORT.md`, and `.gitignore`.